

New to machine learning? Watch a video course to get practical working knowledge of ML using web technologies [View series \(https://goo.gle/learn-webml\)](https://goo.gle/learn-webml)

Save and load models

TensorFlow.js provides functionality for saving and loading models that have been created with the Layers (<https://js.tensorflow.org/api/0.14.2/#Models>) API or converted from existing TensorFlow models. These may be models you have trained yourself or those trained by others. A key benefit of using the Layers api is that the models created with it are serializable and this is what we will explore in this tutorial.

This tutorial will focus on saving and loading TensorFlow.js models (identifiable by JSON files). We can also import TensorFlow Python models. Loading these models are covered in the following two tutorials:

- [Import Keras models](https://www.tensorflow.org/js/tutorials/conversion/import_keras) (https://www.tensorflow.org/js/tutorials/conversion/import_keras)
- [Import Graphdef models](https://www.tensorflow.org/js/tutorials/conversion/import_saved_model) (https://www.tensorflow.org/js/tutorials/conversion/import_saved_model)

Save a tf.Model

tf.Model (<https://js.tensorflow.org/api/0.14.2/#class:Model>) and tf.Sequential (<https://js.tensorflow.org/api/0.14.2/#class:Model>) both provide a function model.save (<https://js.tensorflow.org/api/0.14.2/#tf.Model.save>) that allow you to save the *topology* and *weights* of a model.

- **Topology:** This is a file describing the architecture of a model (i.e. what operations it uses). It contains references to the models's weights which are stored externally.
- **Weights:** These are binary files that store the weights of a given model in an efficient format. They are generally stored in the same folder as the topology.

Let's take a look at what the code for saving a model looks like

```
const saveResult = await model.save('localStorage://my-model-1');
```

A few things to note:

- The `save` method takes a URL-like string argument that starts with a **scheme**. This describes the type of destination we are trying to save a model to. In the example above the scheme is `localStorage://`
- The scheme is followed by a **path**. In the example above the path is `my-model-1`.
- The `save` method is asynchronous.
- The return value of `model.save` is a JSON object that carries information such as the byte sizes of the model's topology and weights.
- The environment used to save the model does not impact which environments can load the model. Saving a model in `node.js` does not prevent it from being loaded in the browser.

Below we will examine the different schemes available.

Local Storage (Browser only)

Scheme: `localStorage://`

```
await model.save('localStorage://my-model');
```

This saves a model under the name `my-model` in the browser's local storage (<https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>). This will persist between refreshes, though local storage can be cleared by users or the browser itself if space becomes a concern. Each browser also sets their own limit on how much data can be stored in local storage for a given domain.

IndexedDB (Browser only)

Scheme: `indexeddb://`

```
await model.save('indexeddb://my-model');
```

This saves a model to the browser's IndexedDB (https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API) storage. Like local storage it persists between refreshes, it also tends to have larger limits on the size of objects stored.

File Downloads (Browser only)

Scheme: `downloads://`

```
await model.save('downloads://my-model');
```

This will cause the browser to download the model files to the user's machine. Two files will be produced:

1. A text JSON file named `[my-model].json`, which carries the topology and reference to the weights file described below.
2. A binary file carrying the weight values named `[my-model].weights.bin`.

You can change the name `[my-model]` to get files with a different name.

Because the `.json` file points to the `.bin` using a relative path, the two files should be in the same folder.

Note: some browsers require users to grant permissions before more than one file can be downloaded at the same time.

HTTP(S) Request

Scheme: `http://` or `https://`

```
await model.save('http://model-server.domain/upload')
```

This will create a web request to save a model to a remote server. You should be in control of that remote server so that you can ensure that it is able to handle the request.

The model will be sent to the specified HTTP server via a POST (<https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/POST>) request. The body of the POST is in the `multipart/form-data` format and consists of two files

1. A text JSON file named `model.json`, which carries the topology and reference to the weights file described below.
2. A binary file carrying the weight values named `model.weights.bin`.

Note that the name of the two files will always be exactly as specified above (the name is built in to the function). This [api doc](https://js.tensorflow.org/api/latest/#tf.io.browserHTTPRequest) (<https://js.tensorflow.org/api/latest/#tf.io.browserHTTPRequest>) contains a Python code snippet that demonstrates how one may use the [flask](http://flask.pocoo.org/) (<http://flask.pocoo.org/>) web framework to handle the request originated from `save`.

Often you will have to pass more arguments or request headers to your HTTP server (e.g. for authentication or if you want to specify a folder that the model should be saved in). You can gain fine-grained control over these aspects of the requests from `save` by replacing the URL string argument in `tf.io.browserHTTPRequest`. This API affords greater flexibility in controlling HTTP requests.

For example:

```
await model.save(tf.io.browserHTTPRequest(
  'http://model-server.domain/upload',
  {method: 'PUT', headers: {'header_key_1': 'header_value_1'}}));
```

Native File System (Node.js only)

Scheme: `file://`

```
await model.save('file:///path/to/my-model');
```

When running on Node.js we also have direct access to the filesystem and can save models there. The command above will save two files to the path specified after the scheme.

1. A text JSON file named `[model].json`, which carries the topology and reference to the weights file described below.
2. A binary file carrying the weight values named `[model].weights.bin`.

Note that the name of the two files will always be exactly as specified above (the name is built in to the function).

Loading a `tf.Model`

Given a model that was saved using one of the methods above, we can load it using the `tf.loadLayersModel` API.

Let's take a look at what the code for loading a model looks like

```
const model = await tf.loadLayersModel('localStorage://my-model-1');
```

A few things to note:

- Like `model.save()`, the `loadLayersModel` function takes a URL-like string argument that starts with a **scheme**. This describes the type of destination we are trying to load a model from.
- The scheme is followed by a **path**. In the example above the path is `my-model-1`.
- The url-like string can be replaced by an object that matches the `IOHandler` interface.
- The `tf.loadLayersModel()` function is asynchronous.
- The return value of `tf.loadLayersModel` is `tf.Model`

Below we will examine the different schemes available.

Local Storage (Browser only)

Scheme: `localstorage://`

```
const model = await tf.loadLayersModel('localstorage://my-model');
```

This loads a model named `my-model` from the browser's local storage (<https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>).

IndexedDB (Browser only)

Scheme: `indexeddb://`

```
const model = await tf.loadLayersModel('indexeddb://my-model');
```

This loads a model from the browser's IndexedDB (https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API) storage.

HTTP(S)

Scheme: `http://` or `https://`

```
const model = await tf.loadLayersModel('http://model-server.domain/download/model.
```

This loads a model from an http endpoint. After loading the `json` file the function will make requests for corresponding `.bin` files that the `json` file references.

Note: This implementation relies on the presence of the **fetch**

(https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch) method, if you are in an environment that does not provide the fetch method natively you can provide a global method names **fetch** (https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch) that satisfies that interface or use a library like **node-fetch** (<https://www.npmjs.com/package/node-fetch>).

Native File System (Node.js only)

Scheme: `file://`

```
const model = await tf.loadLayersModel('file:///path/to/my-model/model.json');
```

When running on Node.js we also have direct access to the filesystem and can load models from there. Note that in the function call above we reference the `model.json` file itself (whereas when saving we specify a folder). The corresponding `.bin` file(s) should be in the same folder as the `json` file.

Loading models with IOHandlers

If the schemes above are not sufficient for your needs you can implement custom loading behavior with an `IOHandler`. One `IOHandler` that TensorFlow.js provides is **tf.io.browserFiles** (<https://js.tensorflow.org/api/latest/#io.browserFiles>) which allows browser users to upload model files in the browser. See the [documentation](https://js.tensorflow.org/api/latest/#io.browserFiles) (<https://js.tensorflow.org/api/latest/#io.browserFiles>) for more information.

Saving and Loading Models with custom IOHandlers

If the schemes above are not sufficient for your loading or saving needs you can implement custom serialization behavior by implementing an `IOHandler`.

An `IOHandler` is an object with a `save` and `load` method.

The `save` function takes one parameter that is a matches the **ModelArtifacts** (<https://github.com/tensorflow/tfjs-core/blob/master/src/io/types.ts#L165>) interface and should return a promise that resolves to a **SaveResult** (<https://github.com/tensorflow/tfjs-core/blob/master/src/io/types.ts#L107>) object.

The `load` function takes no parameters and should return a promise that resolves to a [ModelArtifacts](https://github.com/tensorflow/tfjs-core/blob/master/src/io/types.ts#L165) object. This is the same object that is passed to `save`.

See [BrowserHTTPRequest](https://github.com/tensorflow/tfjs-core/blob/master/src/io/browser_http.ts) for an example of how to implement an `IOHandler`.

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (<https://creativecommons.org/licenses/by/4.0/>), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (<https://www.apache.org/licenses/LICENSE-2.0>). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (<https://developers.google.com/site-policies>). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2022-11-01 UTC.